Faculty of Electrical and Computer Engineering

Workshop Testing Cycle

Member:

Leonardo Zambrano Toala - ltzambra@espol.edu.ec

Teacher:

Dr. Carlos Mera Gómez

Guayaquil – Ecuador

2026

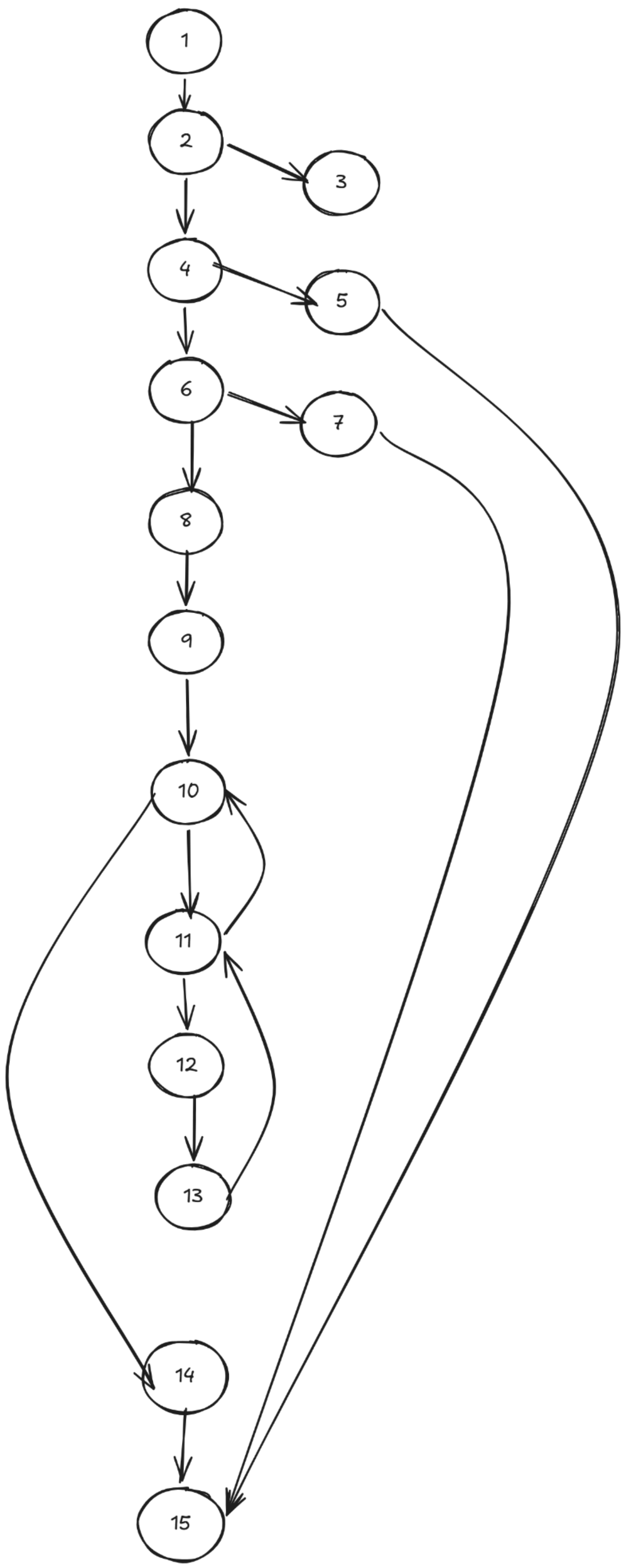
Part 1: Prepare the project

```
PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m venv .venv
PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> .venv\Scripts\activate
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pip install -r requirements.txt -e .
Obtaining file:///C:/Users/leoza/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting pytest>=8.0
  Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 386.5/386.5 kB 3.0 MB/s eta 0:00:00
Collecting pytest-cov>=5.0
  Downloading pytest_cov-7.1.0-py3-none-any.whl (22 kB)
Collecting colorama>=0.4
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting iniconfig>=1.0.1
  Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Collecting packaging>=22
  Using cached packaging-26.2-py3-none-any.whl (100 kB)
Collecting pluggy<2,>=1.5
  Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting pygments>=2.7.2
  Using cached pygments-2.20.0-py3-none-any.whl (1.2 MB)
Collecting coverage[toml]>=7.10.6
  Downloading coverage-7.15.3-cp311-cp311-win_amd64.whl (224 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 224.9/224.9 kB 6.9 MB/s eta 0:00:00
Building wheels for collected packages: roman
  Building editable for roman (pyproject.toml) ... done
  Created wheel for roman: filename=roman-0.1.0-0.editable-py3-none-any.whl size=1257 sha256=3b2d9663bb4bc879059c452ae50a854c8174ab1e626037bc1c9b960962d396ba
  Stored in directory: C:\Users\leoza\AppData\Local\Temp\pip-ephem-wheel-cache-fcvxvuxn\wheels\61\57\16\e798f3977c041b34d1295a61271179d143a220bc2ea4074235
Successfully built roman
Installing collected packages: roman, pygments, pluggy, packaging, iniconfig, coverage, colorama, pytest, pytest-cov
Successfully installed colorama-0.4.6 coverage-7.15.3 iniconfig-2.3.0 packaging-26.2 pluggy-1.6.0 pygments-2.20.0 pytest-9.1.1 pytest-cov-7.1.0 roman-0.1.0

[notice] A new release of pip available: 22.3.1 -> 26.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pytest
============================================= test session starts ==============================================
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 15 items

tests\test_converter.py ...............                                                                  [100%]
```

Part 2: Audit the inherited suite

```
============================================= 15 passed in 0.07s ==============================================
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pytest --cov=roman.converter --cov-branch --cov-report=term-missing
============================================= test session starts ==============================================
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 15 items

tests\test_converter.py ...............                                                                  [100%]

=============================================== tests coverage ================================================
_______________________________ coverage: platform win32, python 3.11.3-final-0 _______________________________

Name                      Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------
src\roman\converter.py       68     24     34      9    64%   42, 44, 46, 58, 61, 64, 72-74, 79, 83, 88, 92-96, 100-104, 108, 112
---------------------------------------------------------------------
TOTAL                        68     24     34      9    64%
============================================= 15 passed in 0.12s ==============================================
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
```

Part 3: Test at the unit level



Compute cyclomatic complexity V(G) = E - N + 2, and report the values of E and N you used. The result must match your graph.

- **Number of Edges (E):** 19
- **Number of Nodes (N):** 15

Using the formula for cyclomatic complexity:

V(G) = E - N + 2

V(G) = 19 - 15 + 2

V(G) = 6

List a basis set of V(G) linearly independent paths, writing each one as a sequence of nodes.

- **Path 1:**
  1 -> 2 -> 4 -> 6 -> 8 -> 9 -> 10 -> 14 -> 15
- **Path 2:**
  1 -> 2 -> 3 -> 15
- **Path 3:**
  1 -> 2 -> 4 -> 5 -> 15
- **Path 4:**
  1 -> 2 -> 4 -> 6 -> 7 -> 15
- **Path 5:**
  1 -> 2 -> 4 -> 6 -> 8 -> 9 -> 10 -> 11 -> 10 -> 14 -> 15
- **Path 6:**
  1 -> 2 -> 4 -> 6 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 11 -> 10 -> 14 -> 15

Build the definition-use table for the variables of to_roman, marking each use as c-use or p-use. The variable remaining is redefined inside the loop, so include the pairs that this redefinition creates

	Category		
LINE	**defintion**	**c-use**	**p-use**
40	n		
41			n
42			
43			n
44			
45			n
46			
47	out		
48	remaining	n	
49	value, symbol		
50			remaining, value
51		out, symbol	
52	remaining	remaining, value	
53		out	

Identify du-pairs and their use (p- or c-)

Definition use pair	Variable(s)	
start line -> end line		
40->41		n
40->43		n
40->45		n

40->48	n	
47->51	out	
47->53	out	
48->50		remaining
48->52	remaining	
52->50		remaining
52->52	remaining	
49->50		value
49->52	value	
49->51	symbol	

Write unit tests until branch coverage of src/roman/converter.py reaches at least 85% coverage of what was captured in part 2.

```python
def test_to_roman_rejects_non_integer():
    with pytest.raises(RomanError, match="value must be an integer"):
        to_roman("10")


def test_to_roman_rejects_boolean():
    with pytest.raises(RomanError, match="value must be an integer"):
        to_roman(True)


def test_to_roman_rejects_value_below_range():
    with pytest.raises(RomanError, match="value must be >= 1"):
        to_roman(0)


def test_to_roman_rejects_value_above_range():
    with pytest.raises(RomanError, match="value must be <= 3999"):
        to_roman(4000)


def test_to_roman_executes_multiple_loop_iterations():
    assert to_roman(3888) == "MMMDCCCLXXXVIII"


def test_from_roman_rejects_non_string():
    with pytest.raises(RomanError, match="value must be a string"):
        from_roman(10)


def test_from_roman_rejects_empty_string():
    with pytest.raises(RomanError, match="empty string"):
        from_roman("")


def test_from_roman_rejects_invalid_character():
    with pytest.raises(RomanError, match="invalid roman character"):
        from_roman("A")


def test_from_roman_accepts_valid_subtractive_pair():
    assert from_roman("IX") == 9
```

```
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pytest --cov=roman.converter --cov-branch --cov-report=term-missing
========================================================================== test session starts ===========================================================================
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 29 items

tests\test_converter.py .............................                                                                                                              [100%]

============================================================================= tests coverage =============================================================================
____________________________________________________________ coverage: platform win32, python 3.11.3-final-0 _____________________________________________________________

Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------
src\roman\converter.py        68      6     34      0    90%   88, 92-96
----------------------------------------------------------------------
TOTAL                         68      6     34      0    90%
========================================================================== 29 passed in 0.18s ==========================================================================
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
```

Part 4: Test at the integration level

Unit tests exercise one function at a time. Integration tests combine two or more units and verify that they work together as a group.

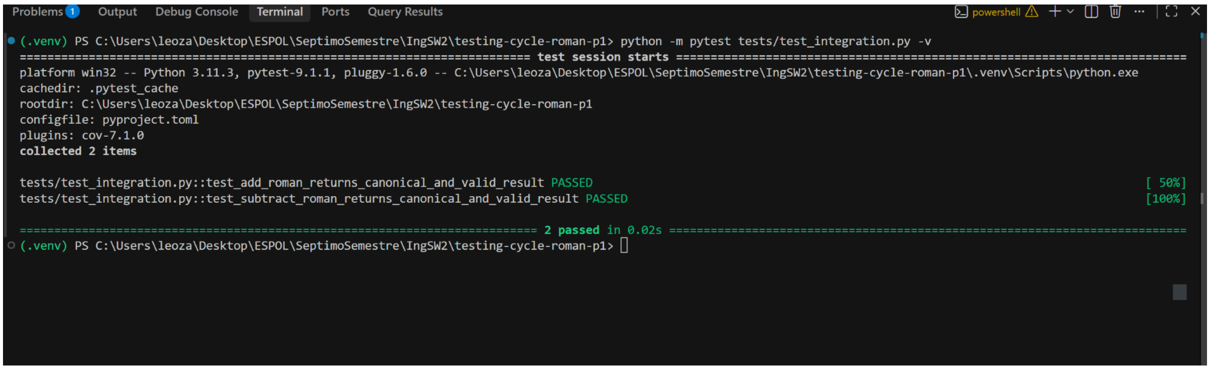
Read section 7 of the specification. The functions add_roman and subtract_roman are built on top of from_roman and to_roman, and their result must be accepted by is_valid_roman.

```
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m pytest tests/test_integration.py -v
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
plugins: cov-7.1.0
collected 2 items

tests/test_integration.py::test_add_roman_returns_canonical_and_valid_result FAILED                                                                                [ 50%]
tests/test_integration.py::test_subtract_roman_returns_canonical_and_valid_result PASSED                                                                           [100%]

=============================================================================== FAILURES ================================================================================
___________________________________________________________ test_add_roman_returns_canonical_and_valid_result ___________________________________________________________

    def test_add_roman_returns_canonical_and_valid_result():
        result = add_roman("II", "II")

>       assert result == "IV"
E       AssertionError: assert 'IIII' == 'IV'
E
E         - IV
E         + IIII

tests\test_integration.py:12: AssertionError
======================================================================== short test summary info ========================================================================
FAILED tests/test_integration.py::test_add_roman_returns_canonical_and_valid_result - AssertionError: assert 'IIII' == 'IV'
====================================================================== 1 failed, 1 passed in 0.10s ======================================================================
```

The integration test verifies the collaboration between `add_roman`, `from_roman`, `to_roman`, and `is_valid_roman`. The test executes `add_roman("II", "II")`. Both operands are converted from Roman numerals to integers, added, and converted back to a Roman numeral. The produced result is then passed to `from_roman` and `is_valid_roman`. The expected canonical result according to section 7 of the specification is `IV`. However, the system returned `IIII`. The inherited unit tests did not detect this defect because they tested simple values such as I, II, III and V independently. They did not test the interaction of the arithmetic and conversion functions for the subtractive value four.

Correction and verification.

The defect was located in the `_PAIRS` collection used by `to_roman`. The Roman numeral `IV` was incorrectly associated with the integer value 5 instead of 4:



After applying the correction, the integration test was executed again. The function add_roman("II", "II") returned the canonical result IV.

The result was successfully converted back to the integer 4 by from_roman, and it was accepted as valid by is_valid_roman.

All integration tests passed after the correction. The complete regression suite was also executed to verify that the change did not break the inherited tests.

Part 5: Test at the acceptance level

Write three acceptance criteria in Given / When / Then form, taken from ESPECIFICACION.md.

Acceptance Criterion 1 Given a valid canonical Roman numeral with leading and trailing whitespace When the user converts " IV " using `from_roman` Then the system returns the integer 4.

Acceptance Criterion 2 Given the noncanonical Roman numeral "IIII" When the user attempts to convert it using `from_roman` Then the system raises `RomanError` because the canonical representation of 4 is "IV".

Acceptance Criterion 3 Given a value that is not a string When the user validates the integer 123 using `is_valid_roman` Then the system returns `False` and does not raise an exception.

Implement each criterion as a test. At least one of them must fail on code that already reports 85% branch coverage.

tests > test_acceptance.py > test_acceptance_validation_returns_false_for_non_string

```python
import pytest

from roman.converter import (
    RomanError,
    from_roman,
    is_valid_roman,
)


def test_acceptance_trims_leading_and_trailing_whitespace():
    """
    Given a valid canonical Roman numeral with outer whitespace
    When the user converts it using from_roman
    Then the system returns the corresponding integer
    """
    assert from_roman("  IV  ") == 4


def test_acceptance_rejects_noncanonical_roman_numeral():
    """
    Given the noncanonical Roman numeral IIII
    When the user attempts to convert it using from_roman
    Then the system raises RomanError
    """
    with pytest.raises(RomanError):
        from_roman("IIII")


def test_acceptance_validation_returns_false_for_non_string():
    """
    Given a value that is not a string
    When the user validates it using is_valid_roman
    Then the system returns False without raising an exception
    """
    assert is_valid_roman(123) is False
```

```
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m pytest --cov=roman.converter --cov-branch --cov-report=term-missing
=========================================================================== test session starts ===========================================================================
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 34 items

tests\test_acceptance.py FF.                                                                                                                                        [  8%]
tests\test_converter.py .............................                                                                                                               [ 94%]
tests\test_integration.py ..                                                                                                                                        [100%]

================================================================================ FAILURES ================================================================================
_______________________________________________________ test_acceptance_trims_leading_and_trailing_whitespace _______________________________________________________

    def test_acceptance_trims_leading_and_trailing_whitespace():
        """
        Given a valid canonical Roman numeral with outer whitespace
        When the user converts it using from_roman
        Then the system returns the corresponding integer
        """
>       assert from_roman("  IV  ") == 4
               ^^^^^^^^^^^^^^^^^^^^

tests\test_acceptance.py:16:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

s = '  IV  '

    def from_roman(s):
        if not isinstance(s, str):
            raise RomanError("value must be a string")
```

```
Problems 2   Output   Debug Console   Terminal   Ports   Query Results                                                              powershell
        text = s.upper()
        if text == "":
            raise RomanError("empty string is not a roman numeral")
        for ch in text:
            if ch not in _SINGLE:
>               raise RomanError("invalid roman character: " + ch)
E               roman.converter.RomanError: invalid roman character:

src\roman\converter.py:64: RomanError
__________________________________________________________ test_acceptance_rejects_noncanonical_roman_numeral __________________________________________________________

    def test_acceptance_rejects_noncanonical_roman_numeral():
        """
        Given the noncanonical Roman numeral IIII
        When the user attempts to convert it using from_roman
        Then the system raises RomanError
        """
>       with pytest.raises(RomanError):
E       Failed: DID NOT RAISE RomanError

tests\test_acceptance.py:25: Failed
============================================================================ tests coverage ============================================================================
___________________________________________________________ coverage: platform win32, python 3.11.3-final-0 ___________________________________________________________

Name                        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------
src\roman\converter.py         68      6     34      0    90%   88, 92-96
-----------------------------------------------------------------------
TOTAL                          68      6     34      0    90%
======================================================================= short test summary info ========================================================================
FAILED tests/test_acceptance.py::test_acceptance_trims_leading_and_trailing_whitespace - roman.converter.RomanError: invalid roman character:
FAILED tests/test_acceptance.py::test_acceptance_rejects_noncanonical_roman_numeral - Failed: DID NOT RAISE RomanError
===================================================================== 2 failed, 32 passed in 0.26s =====================================================================
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
```

The three acceptance criteria were implemented as functional tests derived from the specification. The whitespace acceptance test failed because `from_roman` did not remove leading and trailing whitespace before processing the input. The noncanonical numeral test failed because `from_roman("IIII")` returned the integer 4 instead of raising `RomanError`. The non-string validation test passed because `is_valid_roman(123)` returned `False` without propagating an exception. These failures occurred even though branch coverage was already at least 85%. Branch coverage only indicates that the branches of the implementation were executed. It does not verify that the observed results comply with the functional specification.

Part 6: Iteration

20. Compare the results of each test and in case it's needed, fix three defects, and leave the whole suite green. Do not modify or delete the 15 inherited tests.

21. Commit each fix separately. State in the message the level of testing that found the defect.

fix(unit): correct boundary operator in per spec section N

fix(acceptance): implement per spec section N

```
leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git commit -m "fix(acceptance): trim outer whitespace per spec section 3"
[main 792fd36] fix(acceptance): trim outer whitespace per spec section 3
 1 file changed, 63 insertions(+), 1 deletion(-)

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git commit -m "fix(acceptance): trim outer whitespace per spec section 3"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git add .

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git commit -m "fix(integration): correct IV value per spec section 7"
[main cf65dae] fix(integration): correct IV value per spec section 7
 1 file changed, 3 insertions(+)

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git add .

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git commit -m "fix(acceptance): trim outer whitespace per spec section 3"
[main 0964241] fix(acceptance): trim outer whitespace per spec section 3
 1 file changed, 3 insertions(+)

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git add .

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git commit -m "fix(acceptance): reject noncanonical numerals per spec section 4"
[main cc3cbae] fix(acceptance): reject noncanonical numerals per spec section 4
 1 file changed, 4 insertions(+)

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
$ git push
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 8 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (20/20), 2.03 KiB | 691.00 KiB/s, done.
Total 20 (delta 12), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 3 local objects.
To https://github.com/leozam02/testing-cycle-roman-p1.git
   dc61cf2..cc3cbae  main -> main

leoza@LeoLaptop MINGW64 ~/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1 (main)
```

```
Problems  Output  Debug Console  Terminal  Ports  Query Results                                   powershell

(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m pytest --cov=roman.converter --cov-branch --cov-report=term-missing
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 34 items

tests\test_acceptance.py ...                                                                                                                                    [  8%]
tests\test_converter.py .............................                                                                                                           [ 94%]
tests\test_integration.py ..                                                                                                                                    [100%]

============================================================================ tests coverage =============================================================================
____________________________________________________________ coverage: platform win32, python 3.11.3-final-0 ____________________________________________________________

Name                      Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------
src\roman\converter.py      103      5     58      4    94%   100, 120, 132, 158, 161
---------------------------------------------------------------------
TOTAL                       103      5     58      4    94%
========================================================================== 34 passed in 0.45s ===========================================================================
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
```

The results from the unit, integration and acceptance tests were compared before modifying the implementation.

The unit tests provided the required structural coverage. The integration and acceptance tests revealed three defects.

Defect 1: Incorrect representation of IV

Testing level: Integration testing.

The integration test found that `add_roman("II", "II")` returned `IIII` instead of the canonical result `IV`. The defect was caused by the following incorrect pair:

(5, "IV")

It was corrected to: (4, "IV")

After the correction, the integration test passed.

Defect 2: Outer whitespace was not removed

Testing level: Acceptance testing

The acceptance test found that from_roman(" IV ") raised RomanError, although the specification requires leading and trailing whitespace to be ignored.

The implementation:

text = s.upper()

was corrected to:

text = s.strip().upper()

After the correction, the function returned the integer 4.

Defect 3: Noncanonical numeral was accepted

Testing level: Acceptance testing

The acceptance test found that from_roman("IIII") returned 4 instead of raising RomanError.

Canonical-format validation was added. After the correction, from_roman("IIII") raises RomanError and is_valid_roman("IIII") returns False.

Final verification

After correcting the three defects, the complete test suite was executed again.

- All 34 tests passed.
- No test failed.
- Branch coverage remained at least 85%.
- The 15 inherited tests were not modified or deleted.
- Each defect was corrected in a separate commit.